Efficient Verification Tips

Aman Arora



Mixed

- Checking inside stimulus
  - Self-checking testcases
- Coverage inside stimulus
  - Sample covergroups inside directed tests
- Checking inside coverage
  - Illegal bins
- Coverage within checking
  - Cover properties

What Happens Where?

- Stimulus
  - Goes in sequences.
  - May have some basic bringup in an initial block in the testbench
- Coverage
  - Can go in monitors
  - Separate coverage classes that receive transactions from monitors
  - Can also be in interfaces (depends on the type of coverage you wanna write)
- Checking
  - Self-checking stimulus
  - Some basic checking in monitors
  - Can go in scoreboards (separate components)
  - Can also be in interfaces (depends on the type of checks you wanna write)

Plusargs

- Pass options to a simulation.
- ./simv +VERBOSE +NUM_PACKETS=100 +TEST=memory_test
- Reading them inside the code

```
if ($test$plusargs("VERBOSE"))
    $display("Verbose mode enabled");
```

```
int num_packets;
string test_name;

initial begin
    if (!$value$plusargs("NUM_PACKETS=%d", num_packets))
        num_packets = 10;     // default

    if (!$value$plusargs("TEST=%s", test_name))
        test_name = "basic";
end
```

- Usecase: Build once, run many times

XMRs

- Access a signal/variable in another module using its hierarchical path

```
// Hierarchical references into the DUT
$display("PC = %h", top.cpu0.pc);

if (top.cpu0.stall)
    $display("CPU stalled");
```

- Issue
  - Brittle (things can break when something changes)
  - Not so good for reuse
  - Use of `defines or macros

PLI and DPI

- PLI vs. DPI
  - PLI = Programming Language Interface
  - DPI = Direct Programming Interface
- tf, acc, vpi
  - User-defined system tasks and functions, Accessing and manipulating objects in Verilog design hierarchy, Controlling simulation
- C files
  - Provide them on the VCS command line
  - Or compile them into a shared object library, and then provide it on the VCS command line

Message Severity

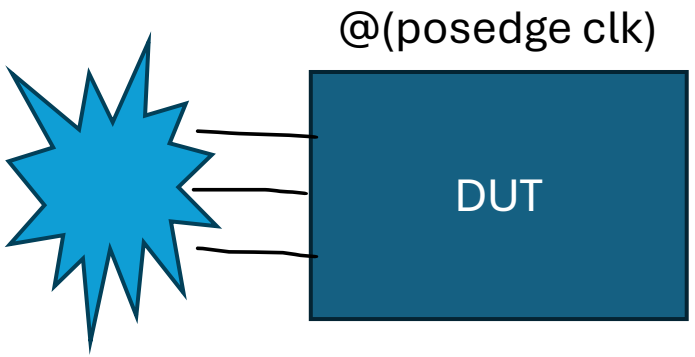
- $info, UVM_INFO
- $warning, UVM_WARNING
- $error, UVM_ERROR
- $fatal, UVM_FATAL

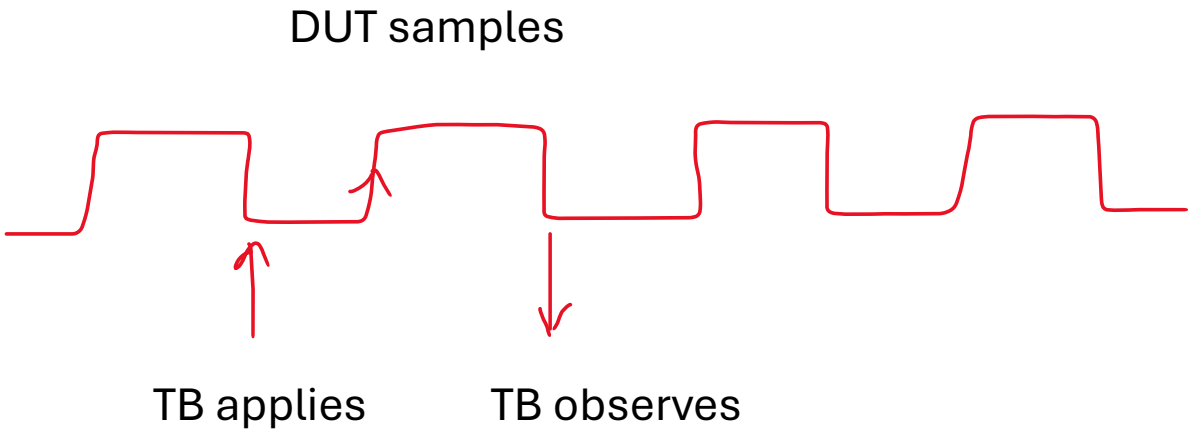
White-box vs. Black-box verification

- Black-box
  - Verification engineer only knows the specification
  - Only look at input and output signals
  - Finding spec. violations
  - Hard to localize failures or do debugging
- White-box
  - Verification engineer has access to the full RTL
  - Can look at internal signals as well
  - Finding implementation bugs
  - Can have common-mode issues (RTL and TB have the same bug)

Gray-Box

Time Offset in Applying Inputs and Observing Outputs





Clocking Blocks

- Defines when a testbench samples and drives interface signals relative to a clock, avoiding race conditions between the DUT and testbench.

```
interface dut_if(input logic clk);

    logic req;
    logic [7:0] data;
    logic done;
    logic [7:0] result;

    clocking cb @(posedge clk);
        default input #1ns output #2ns;

        output req, data;     // TB drives these 2 ns AFTER posedge
        input  done, result;  // TB samples these 1 ns BEFORE posedge
    endclocking

endinterface
```

```
initial begin
    @(intf.cb);

    intf.cb.req  <= 1'b1;
    intf.cb.data <= 8'h42;

    $display("done = %b", intf.cb.done);
end
```

Gate-Level Simulations

- Simulate synthesized gate-level netlist instead of RTL
- Catches
  - RTL vs. synthesized-netlist mismatch
  - X-propagation / initialization problems hidden by RTL assumptions
  - Reset and power-up issues
  - Timing-dependent behavior
  - Issues involving gated clocks (or any other logic that is introduced during synthesis)
- Types
  - Zero delay simulation
  - Timing simulation
    - SDF (Standard Delay Format)

Force and Release Statements

- Override DUT signals
- force dut.blah.blah.sig1 = 1’b1;
- release dut.blah.blah.sig1
- Common uses:
  - Fault injection
  - Create hard-to-reach conditions (corner cases)
  - Debugging

Error injection for error scenarios

- Special tests to inject errors
  - Flip a memory bit
  - Corrupt a packet
  - Force invalid opcode
  - Drop a response
- Ensure that the DUT behaves correctly
  - Can recover from the error or flag an interrupt or something
- How?
  - Use XMRs
  - Disable checks
  - Disable constraints

Walkthrough of OpenTitan

- https://opentitan.org/book/index.html